Executive Summary

The integration of artificial intelligence into the software development life cycle (SDLC) represents a paradigm shift, transitioning the developer's role from raw code generation to architectural orchestration and system restoration. However, recent empirical data, including the 2025 DevOps Research and Assessment (DORA) State of AI-assisted Software Development report, reveals a critical paradox: while AI significantly accelerates code creation, it simultaneously acts as an amplifier of existing systemic dysfunctions.[1] In brownfield enterprise environments—characterized by accumulated technical debt, incomplete documentation, and fragile testing harnesses—deploying AI solely for feature velocity accelerates the generation of technical debt and degrades system stability.[1]

This comprehensive research report provides the architectural blueprint and pedagogical foundation for a rigorous five-day enterprise training program. Developed with a specific operational context in mind—midwestern enterprise hubs such as Edina, Minnesota, heavily utilizing Microsoft Azure,.NET 8, and emerging Python (FastAPI/Django) workloads—the curriculum is designed specifically for software engineers, tech leads, and architects managing legacy systems. The core thesis posits that the most strategic application of AI coding assistants, specifically the Anthropic Claude Code CLI, is not merely writing new features, but systematically identifying, isolating, and paying down the architectural and technical debt that impedes safe change.[3]

By establishing a clear taxonomy of "What Good Looks Like" (WGLL) and deploying structured, AI-assisted workflows (e.g., characterization testing, automated dependency mapping, and deterministic refactoring via Git worktrees), engineering teams can reclaim maintainability. The resulting course design is platform-neutral regarding underlying principles but highly concrete in its application, offering specific implementation paths for.NET and Python workloads deployed within the Azure ecosystem.[5] The report provides a cohesive narrative, exhaustive slide materials, hands-on exercises utilizing the Claude Code CLI, and reusable instructor assets to ensure the training translates into immediate organizational impact.

Recommended Course Narrative

The five-day course must eschew disconnected tool tutorials in favor of a cohesive narrative arc that mirrors the logical progression of software modernization. The narrative follows a "Discover, Define, De-risk, Deliver, and Democratize" trajectory, ensuring that students build a compounding mental model of AI-assisted engineering.

1. **The Amplifier Effect (Day 1):** The course begins by confronting the reality of brownfield systems. Students learn that AI operating without boundaries generates liabilities. The narrative focuses on using AI as an archaeological tool to map undocumented domains, generate architectural clarity (C4 models, Architecture Decision Records), and establish reproducible developer environments as a baseline. The core lesson is that AI must first

understand the system before it can change it.

2. **The Safety Harness (Day 2):** Before any legacy code is altered, behavior must be codified. The narrative shifts to testability, teaching students how to use AI to rapidly generate characterization tests that lock in existing behavior, thereby neutralizing the risk of silent semantic drift during subsequent refactoring. This day establishes the rule that testing precedes all AI modification.
3. **The Operational Lens (Day 3):** Code quality is meaningless if production behavior is opaque. The story moves to observability, emphasizing OpenTelemetry and Azure Application Insights. Students learn how AI can instrument code and interpret operational telemetry (logs, metrics, traces) to identify performance bottlenecks and runtime debt, shifting the team from reactive monitoring to proactive investigation.
4. **The Incremental Scalpel (Day 4):** With safety nets (tests) and visibility (observability) in place, the narrative addresses safe change. Students learn to orchestrate Claude Code CLI subagents to execute bounded, incremental refactors—avoiding the catastrophic "big bang" rewrite in favor of the Strangler Fig pattern and isolated Git worktrees.
5. **The Institutional Memory (Day 5):** The course concludes by transitioning from individual heroics to team-level discipline. The narrative explores embedding AI guardrails directly into the repository via CLAUDE.md files, custom Claude Skills, and CI/CD hooks, ensuring that the definition of "good" survives beyond the lifespan of the training class and scales across the enterprise.

Improved Taxonomy of “What Good Looks Like”

The initial seed list of project health indicators is comprehensive but overly fragmented. To facilitate effective teaching, these indicators have been synthesized into five coherent pillars of Software Delivery Excellence. This taxonomy provides the structural framework for the course, aligning tactical practices with strategic business outcomes.

Pillar 1: Architectural and Boundary Clarity

- **Why it matters:** In enterprise brownfield systems, tribal knowledge decays, leaving a massive cognitive load for developers.[7] Without clear module boundaries and explicit data contracts, changes generate unpredictable downstream ripple effects, making the system brittle and resistant to evolution.
- **What failure looks like:** "Spaghetti" dependencies, circular references, missing or outdated README files, and "God classes" that violate single-responsibility principles.[8] Failure is also evident when developers must consult multiple disparate wikis to understand basic system flow.
- **What technical debt looks like:** Hard-coded business rules dispersed across UI and database layers; undocumented integrations; absence of Architecture Decision Records (ADRs); missing service boundary definitions.

- **How AI tools help:** The Claude Code CLI can analyze thousands of files to map dependencies, reverse-engineer domain contexts, and generate accurate C4 diagrams and ADRs reflecting the *actual* implemented architecture, rather than the aspirational one.[9] AI can summarize vast repositories to provide rapid onboarding context.
- **Measurable signals of progress:** Reduced developer onboarding time; presence of up-to-date ARCHITECTURE.md and CLAUDE.md files; decoupled deployment pipelines for discrete services; clear API contracts documented via OpenAPI/Swagger.

Pillar 2: Environment Reproducibility and Supply Chain Hygiene

- **Why it matters:** The "it works on my machine" anti-pattern destroys velocity. A reproducible environment guarantees that all developers, CI/CD runners, and AI agents operate under identical constraints.[11] This is critical for enterprise Azure shops relying on complex identity and networking configurations.
- **What failure looks like:** Multi-day onboarding processes involving manual software installations; mismatched framework versions between local and production environments; unmanaged third-party dependencies leading to supply-chain vulnerabilities.[13]
- **What technical debt looks like:** Obsolete build scripts; lack of containerization (Docker/DevContainers); manual database provisioning requirements for local runs; hardcoded local connection strings instead of Azure Managed Identities.[14]
- **How AI tools help:** AI can seamlessly translate imperative setup instructions into declarative Infrastructure as Code (IaC), generate devcontainer.json specifications, and analyze requirements.txt or .csproj files to identify deprecated or vulnerable transitive dependencies.[11] AI can also scaffold Azure Developer CLI (azd) templates to ensure local-to-cloud parity.[16]
- **Measurable signals of progress:** Time-to-first-successful-local-build drops to under 15 minutes; 100% parity between local, QA, and Production OS/framework runtimes; automated vulnerability scans passing in CI pipelines.

Pillar 3: Testability and State Determinism

- **Why it matters:** High-confidence deployments require deterministic validation. If a system cannot be easily tested in isolation, developers will avoid refactoring it, allowing debt to compound exponentially.[17]
- **What failure looks like:** Flaky tests that fail randomly; tests that require a shared, mutating staging database; inability to stub external API dependencies.[18]
- **What technical debt looks like:** Tight coupling to global state; lack of database schema versioning (e.g., Entity Framework Migrations, Alembic); monolithic end-to-end tests used as a substitute for fast unit tests; manual test data generation.
- **How AI tools help:** AI assistants excel at generating mock data, writing interface abstractions to decouple hard dependencies, and rapidly scaffolding characterization tests to document the exact current behavior of legacy modules before modification.[19] AI can generate comprehensive boundary value test cases that human developers frequently

overlook.
- **Measurable signals of progress:** High mutation testing scores; execution of the full local test suite in under 5 minutes; automated database teardown and recreation per test run; isolated dependency mocking.

Pillar 4: Observability and Operational Confidence

- **Why it matters:** Modern distributed cloud systems fail in unpredictable ways. Observability shifts engineering from reactive monitoring (knowing a system is down) to proactive investigation (knowing *why* it is down and where the bottleneck resides).[21]
- **What failure looks like:** Silent failures; reliance on SSHing into production servers to read text logs; inability to trace a single user request across multiple microservices or Azure Functions.
- **What technical debt looks like:** Inconsistent log formatting; lack of correlation IDs (W3C Trace Context); absence of Service Level Indicators (SLIs) or business metrics in the telemetry stream; missing dashboards.[22]
- **How AI tools help:** AI can automate the insertion of OpenTelemetry boilerplate across.NET and Python applications, analyze stack traces against repository code to pinpoint root causes, and generate KQL (Kusto Query Language) queries for Azure Application Insights.[5] AI agents can also be deployed to continuously monitor and summarize error logs.
- **Measurable signals of progress:** Reduced Mean Time To Recovery (MTTR); comprehensive distributed tracing across network boundaries; alerts tied to user-centric Service Level Objectives (SLOs) rather than generic CPU utilization metrics.[24]

Pillar 5: Safe Change and Progressive Delivery

- **Why it matters:** The ability to release software safely is the ultimate constraint on value delivery. Changes must be isolated, verified, and easily reversible to foster an engineering culture of continuous improvement.
- **What failure looks like:** Manual, late-night deployment ceremonies; "big bang" cutovers; hard-coded feature toggles requiring full redeployments to alter.[25]
- **What technical debt looks like:** Missing CI/CD pipelines; lack of semantic versioning; secrets and connection strings committed directly to source control; monolithic repositories lacking independent deployment capabilities.[8]
- **How AI tools help:** AI agents can draft complex CI/CD YAML configurations (GitHub Actions, Azure DevOps Pipelines), analyze pull requests for security flaws and hardcoded secrets prior to merge, and plan incremental "Strangler Fig" deprecation strategies for legacy endpoints.[26]
- **Measurable signals of progress:** Increased deployment frequency; zero-downtime deployments; automated rollback capabilities triggered by observability alarms; low Change Failure Rate (CFR).[28]

5-Day Course Plan

Format: 5 Days, 60 minutes per day.

Target Audience: Enterprise Software Engineers, Tech Leads, and Architects.

Prerequisites: Students must have the Claude Code CLI installed locally and access to a sample brownfield repository (preferably.NET 8 or Python FastAPI/Django).

Structure per Day:

- **00:00 - 00:15:** Theory, Concepts, and Anti-patterns (Slide-driven).
- **00:15 - 00:25:** Instructor Demonstration (Live terminal/IDE usage).
- **00:25 - 00:50:** Hands-on Student Exercise (Workshop format).
- **00:50 - 01:00:** Debrief, Q&A, and Synthesis.

Note on Time Constraints: If the 60-minute session runs short, the instructor should reduce the scope of the hands-on exercise (e.g., providing pre-written boilerplate for the exercise) rather than skipping the final debrief. The synthesis of *why* the AI succeeded or failed is the core learning objective and must not be sacrificed.

Day 1: System Archeology and the Reproducible Baseline

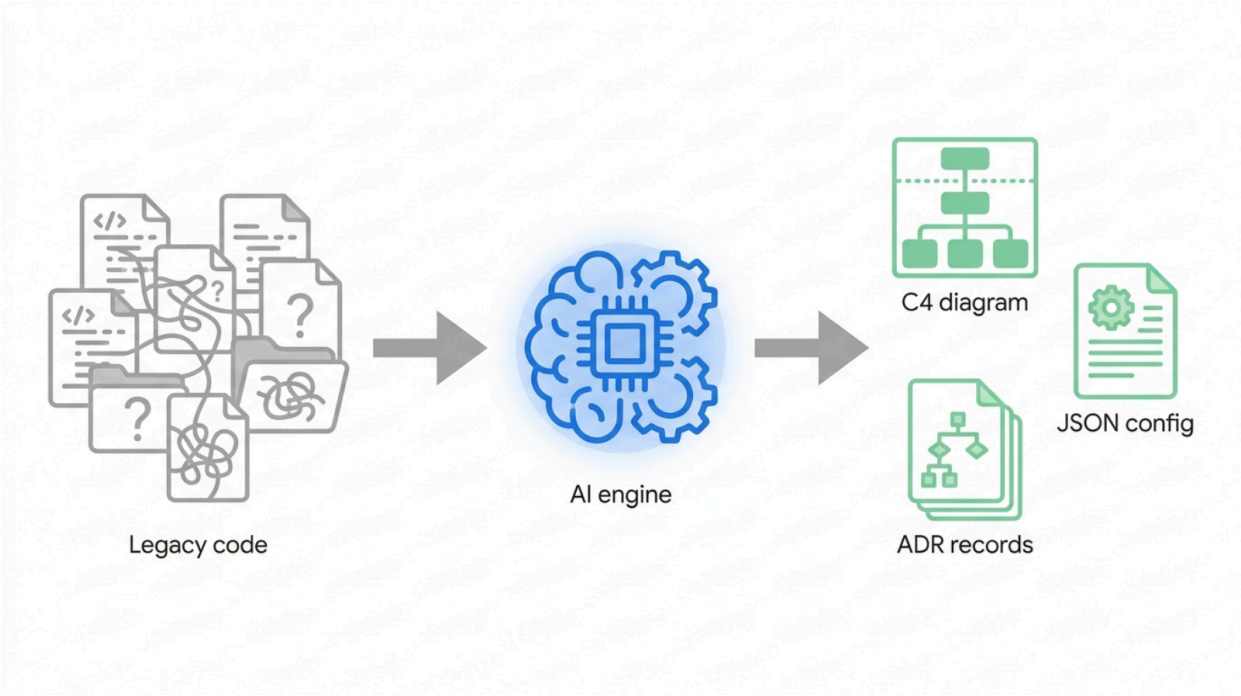
Focus: Pillars 1 & 2 (Architecture Clarity & Reproducibility). Using AI to understand what exists before attempting to change it.

Lesson Outline & Slide-Building Material

- **Lesson Title:** Digging Out: AI for Architectural Discovery and Local Reproducibility.
- **Short Description:** Learn to utilize the Claude Code CLI to map undocumented brownfield systems and codify developer environment setups, eliminating the "works on my machine" friction.
- **Why this belongs here:** Before any refactoring can commence, developers must understand the system boundaries and guarantee that their local environment strictly mirrors production constraints. AI cannot validate its own code if the project cannot build.
- **Learning Objectives:**
  1. Define the parameters of a fully reproducible developer environment.
  2. Leverage the Claude Code CLI to map code dependencies and generate accurate C4 context diagrams.
  3. Generate Architecture Decision Records (ADRs) for legacy systems using AI summarization.
- **Slide-Ready Teaching Points:**
  - **Brownfield reality:** The code is the only source of truth that does not lie;

documentation is often aspirational.[7]
  - **Context Window Leverage:** Modern AI context windows (e.g., 200K tokens) allow CLI agents to ingest entire repositories to identify hidden coupling and circular dependencies.[29]
  - **The Reproducibility Prerequisite:** Reproducibility is mandatory for AI agents. If a human cannot build the project locally via standard commands, the AI agent cannot compile or verify its own code changes.[30]
  - **Codifying Environments:** DevContainers and Azure Developer CLI (azd) templates transform environmental configuration from tribal knowledge into version-controlled, executable code.[11]
  - **The Agent Constitution:** A CLAUDE.md file serves as the "system prompt" for your repository, enforcing architectural rules, build commands, and domain context for all subsequent AI interactions.[31]
- **Common Anti-Patterns / Failure Smells:**
  - "Hero Developer" onboarding: relying on a single senior engineer to help new hires install local dependencies over several days.[32]
  - Outdated Confluence architecture diagrams that reflect original intentions rather than the current implementation.
  - Using AI to write new features before the AI has been explicitly prompted with the project's architectural constraints and boundary limits.
- **Discussion Prompt:** "What is the longest it has ever taken a new engineer on your team to get a local build running successfully, and what were the hidden dependencies that blocked them?"
- **Instructor Demo Idea:** Run claude --plan "Analyze the /services directory in this repository and generate a Level 2 C4 Container diagram using Mermaid.js syntax. Save it to docs/architecture.md." Display the resulting visualization in a Markdown viewer.
- **AI Leverage Notes:** Claude Code's --plan mode is critical for this phase. It forces the AI to read files and emit a hypothesis without modifying the codebase, serving as a powerful, non-destructive exploration tool.[33]
- **Human Judgment Required:** AI may confidently hallucinate architectural boundaries if the codebase is highly entangled. Developers must manually verify the generated C4 diagrams against actual network and database traffic.

AI as an Architectural Archeologist



Before refactoring, AI tools analyze raw brownfield code to generate structural understanding, yielding C4 models, Architecture Decision Records (ADRs), and reproducible environment configurations.

Exercises

- **Exercise Title:** The Repo Interrogation.
- **Learning Goal:** Use the Claude Code CLI to reverse-engineer a legacy module and establish persistent project context via a CLAUDE.md file.
- **Estimated Duration:** 25 minutes.
- **Setup / Prerequisites:** Provide students with a sample.NET 8 or Python Django repository containing intentional technical debt (e.g., tight coupling between business logic and data access, missing setup instructions). Ensure the Claude Code CLI is authenticated.
- **Student Instructions:**
  1. Initialize the Claude Code CLI in the root directory of the sample repository.
  2. Execute the following exploration prompt: claude "Read the source code in the backend directory. Identify the core domain entities, database dependencies, and build commands. Generate a comprehensive CLAUDE.md file outlining project conventions and current architecture."
  3. Review the generated CLAUDE.md. Identify at least one hallucination or incorrect assumption the AI made regarding the architecture, and manually correct the file.

4. Prompt Claude to generate an Architecture Decision Record (ADR) documenting the current state of the database connection management: claude "Based on the codebase, write an ADR in markdown format explaining how database connections are currently managed, highlighting the technical debt involved."

- **Expected Output:** A committed CLAUDE.md file containing accurate build commands and tech stack details, alongside a newly generated ADR documenting existing technical debt.
- **Instructor Notes / Debrief Points:** Emphasize that the AI's first pass will likely miss tribal knowledge. The exercise highlights the absolute necessity of the human expert in correcting and steering the AI's baseline understanding. CLAUDE.md must be treated as a living document.

Day 2: Validation Confidence and Characterization Testing

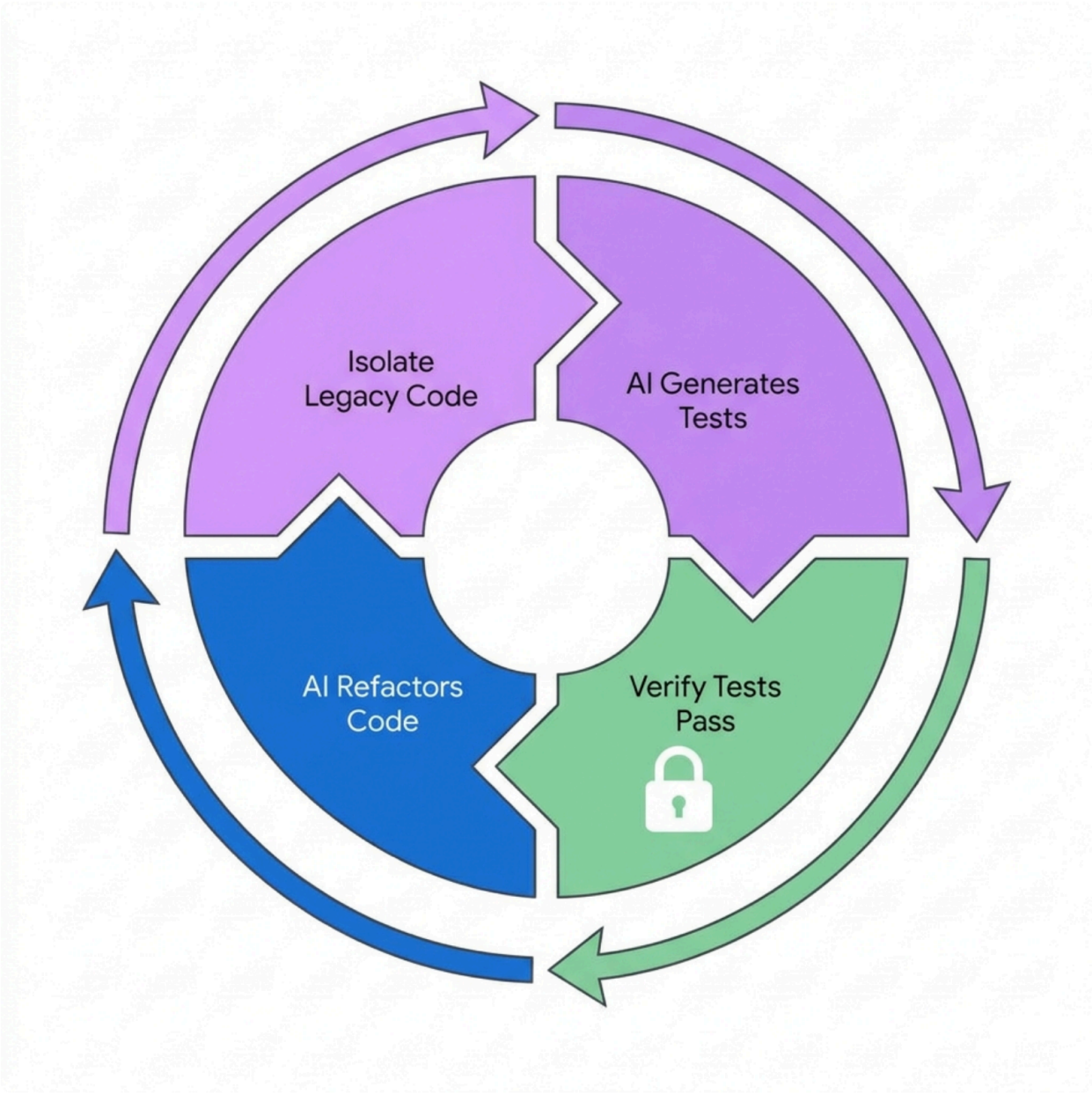
Focus: Pillar 3 (Testability & State Determinism). De-risking legacy code modifications.

Lesson Outline & Slide-Building Material

- **Lesson Title:** The Safety Net: AI-Assisted Characterization Testing.
- **Short Description:** Learn how to lock down the exact behavior of untested legacy code using AI-generated characterization tests before attempting any refactoring or modernization.
- **Why this belongs here:** Modifying brownfield code without a test harness is professional negligence. AI drastically reduces the toil of writing the initial tests required to make subsequent refactoring safe.
- **Learning Objectives:**
  1. Understand the critical difference between Test-Driven Development (TDD) and Characterization Testing in legacy systems.
  2. Prompt the Claude CLI to generate exhaustive edge-case inputs and capture current outputs for legacy functions.
  3. Establish state determinism by utilizing AI to generate database and API mocks.
- **Slide-Ready Teaching Points:**
  - **Legacy Code Definition:** According to Michael Feathers, legacy code is simply "code without tests.".[19]
  - **Characterization vs. Verification:** Characterization tests do not check what the code *should* do; they record what the code *actually* does today, including its bugs and anomalies.[19]
  - **The Golden Rule of AI Refactoring:** Never allow an AI agent to refactor business logic unless a comprehensive test suite passes both before and after the change.[35]
  - **Automating Edge Cases:** Use AI to generate permutations of test data that humans routinely overlook (e.g., boundary values, massive payloads, malformed strings, null injections).

    - **Beware "Testing the Mock":** Ensure the AI does not mock out so much of the internal system that the test always passes regardless of the actual code logic.[37]
- **Common Anti-Patterns / Failure Smells:**
    - Refactoring code and adding tests for that code in the exact same Pull Request (violates the "one PR, one intent" rule and obscures behavioral changes).[36]
    - Targeting 100% generic code coverage instead of focusing on critical business paths and high-risk modules.
    - Accepting AI-generated tests without manually reading the assertions to confirm they align with reality.
- **Discussion Prompt:** "Have you ever 'fixed' an obvious bug in legacy code only to discover later that a downstream system was relying on that exact bug as a feature?"
- **Instructor Demo Idea:** Demonstrate the classic "Gilded Rose" refactoring kata.[38] Run the legacy code, use the Claude CLI to write 20 snapshot tests locking the output, then intentionally break the legacy code to show the tests immediately failing.
- **AI Leverage Notes:** Utilize Claude's ability to read a class and output parameterized tests (e.g., `` in xUnit or @pytest.mark.parametrize in Python) to cover massive functional surface areas quickly.
- **Human Judgment Required:** AI will happily write tests that codify incorrect or dangerous behavior if the legacy code is fundamentally broken. Humans must review the test assertions to ensure they represent actual, acceptable system truths.

The AI-Assisted Safe Refactoring Loop



Characterization tests map existing behavior (including anomalies). Only once behavior is locked by these AI-generated tests is it safe to proceed to refactoring.

Exercises

- **Exercise Title:** The Golden Master.
- **Learning Goal:** Use the Claude Code CLI to generate a characterization test suite for a highly complex, undocumented function, ensuring zero semantic drift during a subsequent

refactor.

- **Estimated Duration:** 30 minutes.
- **Setup / Prerequisites:** Provide a deeply nested, high cyclomatic complexity function (e.g., a massive if/elif/else block calculating pricing or permissions, similar to the Gilded Rose legacy code).
- **Student Instructions:**
  1. Prompt the Claude CLI: claude "Review the legacy pricing calculator in <file>. Write a comprehensive set of characterization tests using [pytest/xUnit] that captures the current output of this function for all edge cases. Do NOT refactor the original code."
  2. Run the generated test suite locally to ensure all tests pass against the current, messy code.
  3. Review the generated tests. Ask Claude to generate a coverage report command to identify any missing execution branches, and prompt Claude to write tests for those specific gaps.
  4. Once coverage is sufficient and behavior is locked, prompt Claude: claude "Now refactor the pricing calculator function to reduce cyclomatic complexity. Ensure all characterization tests still pass."
- **Expected Output:** A clean, refactored function with an accompanying test file that guarantees the business logic was not altered during modernization.
- **Instructor Notes / Debrief Points:** Watch closely for students who allow Claude to change the test files *during* the refactoring phase. Emphasize that the test file must remain completely untouched during step 4; if a test breaks, the refactor must be rolled back.

Day 3: Enterprise Observability and Operational Awareness

Focus: Pillar 4 (Observability & Operational Confidence). Ensuring AI agents and human operators can see system behavior in production.

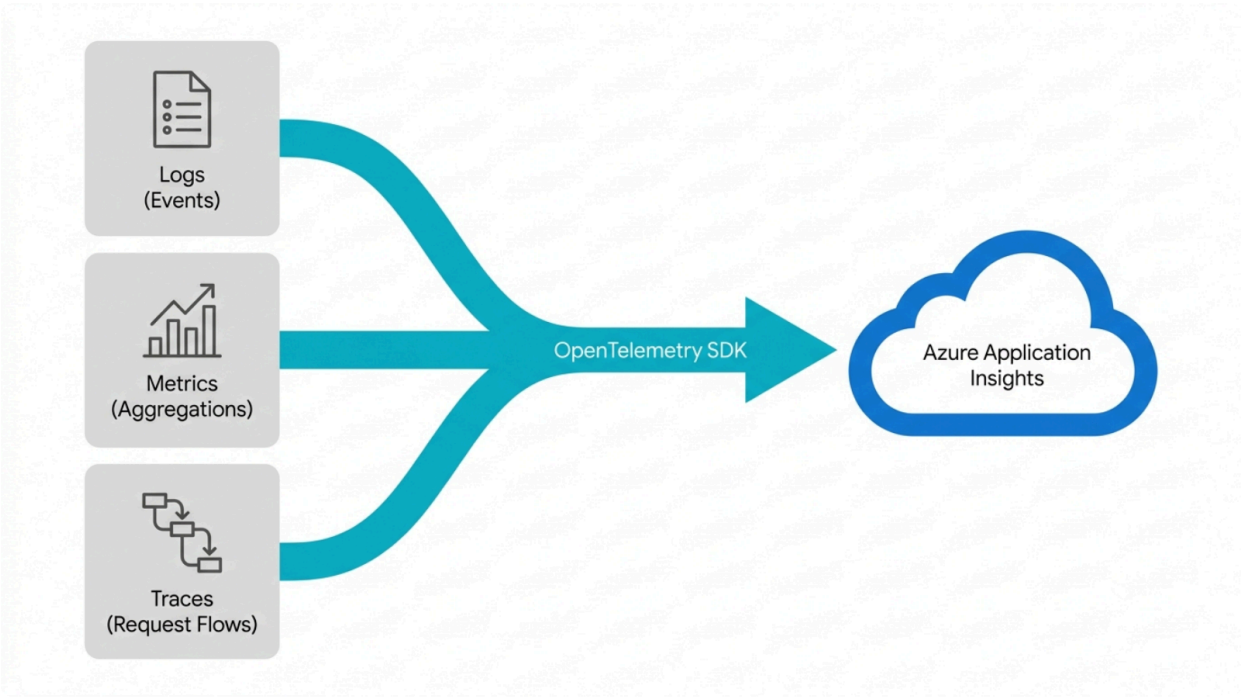
Lesson Outline & Slide-Building Material

- **Lesson Title:** Seeing the Unseen: Integrating OpenTelemetry and Azure Application Insights.
- **Short Description:** Transform black-box legacy applications into fully observable systems using OpenTelemetry, enabling AI agents to diagnose production incidents accurately.
- **Why this belongs here:** Once code is tested and refactored, it must be deployed. If observability is missing, teams cannot verify if their refactor degraded performance or introduced silent errors in production.[22]
- **Learning Objectives:**
  1. Differentiate between traditional monitoring (reactive) and modern observability (investigative).[21]
  2. Implement the three pillars of telemetry: structured logs, metrics, and distributed traces.

3. Utilize AI to rapidly instrument a.NET or Python codebase with OpenTelemetry, routing data to Azure Application Insights.

- **Slide-Ready Teaching Points:**
  - **Monitoring vs. Observability:** Monitoring tells you *if* a system is broken; Observability tells you *why* it is broken and where the fault originated.[21]
  - **AI Needs Telemetry:** AI debugging agents (like Azure Copilot or Claude) are only as effective as the telemetry data they can ingest. Blind systems cripple AI assistance.[6]
  - **Standardization:** OpenTelemetry (OTel) is the vendor-neutral standard for instrumenting.NET and Python applications, preventing vendor lock-in while integrating natively with Azure.[5]
  - **Distributed Tracing:** Implementing Correlation IDs (W3C Trace Context) is mandatory for tracing requests across microservices, Azure Functions, and databases.
  - **Structured Data:** Logs must be structured (e.g., JSON format) so that machines, logging aggregators, and AI agents can query them efficiently.
- **Common Anti-Patterns / Failure Smells:**
  - Relying purely on infrastructure metrics (CPU/Memory) rather than user-centric metrics (e.g., request latency, error rates).
  - "Exception swallowing" in legacy code, where try/catch blocks hide errors that are never logged to the telemetry stream.
  - Logging sensitive Personally Identifiable Information (PII) or hardcoded secrets into plain text telemetry streams.
- **Discussion Prompt:** "How long does it typically take your team to identify the root cause of a production defect? What specific data is usually missing when an incident occurs?"
- **Instructor Demo Idea:** Take a basic Python FastAPI or.NET Minimal API endpoint. Use the Claude Code CLI to inject OpenTelemetry tracing boilerplate. Make a web request and show the resulting Trace in the Azure Portal (Application Map), demonstrating the end-to-end visibility.
- **AI Leverage Notes:** Ask Claude to review a critical file and suggest where custom business metrics or Span attributes should be added to improve visibility. Claude can also generate KQL queries to search the resulting logs in Azure.
- **Human Judgment Required:** AI might suggest overly verbose logging inside tight loops (e.g., logging every iteration of a thousand-item loop), leading to exorbitant ingestion costs in Azure Application Insights. Humans must enforce telemetry sampling strategies and cost-control thresholds.

The Architecture of Observability



OpenTelemetry standardizes the collection of Logs, Metrics, and Traces, funneling structured telemetry into Azure Application Insights to enable AI-assisted root-cause analysis.

Exercises

- **Exercise Title:** Illuminating the Black Box.
- **Learning Goal:** Instrument an existing application with OpenTelemetry using AI assistance to achieve baseline observability.
- **Estimated Duration:** 30 minutes.
- **Setup / Prerequisites:** Provide a basic.NET Web API or Python Flask/FastAPI app without any existing logging infrastructure. Provide a mock or real connection string for Azure Application Insights.
- **Student Instructions:**
  1. Use the Claude CLI to analyze the API endpoints and request instrumentation: claude "I need to add OpenTelemetry tracing and metrics to this app and export it to Azure Monitor. Modify the setup files to include the required [NuGet/Pip] packages and initialization boilerplate."
  2. Review the AI's proposed package additions (e.g., Azure.Monitor.OpenTelemetry.AspNetCore for.NET or azure-monitor-opentelemetry for Python) and apply the changes.[5]
  3. Prompt Claude to add custom Span attributes to a specific business logic function

(e.g., tracking a user_id, order_amount, or tenant_id).
4. Run the application locally, trigger the endpoint, and observe the terminal output to confirm traces are generating correctly.

- **Expected Output:** The application successfully compiles and emits structured trace data encompassing the custom business attributes, ready for ingestion by Azure.
- **Instructor Notes / Debrief Points:** Discuss the difference between auto-instrumentation (which AI can set up instantly) and custom instrumentation (which requires business context). Highlight how having OTel in place allows future AI debugging sessions to consume rich logs rather than guessing at runtime behavior.

Day 4: Safe Refactoring and Incremental Change

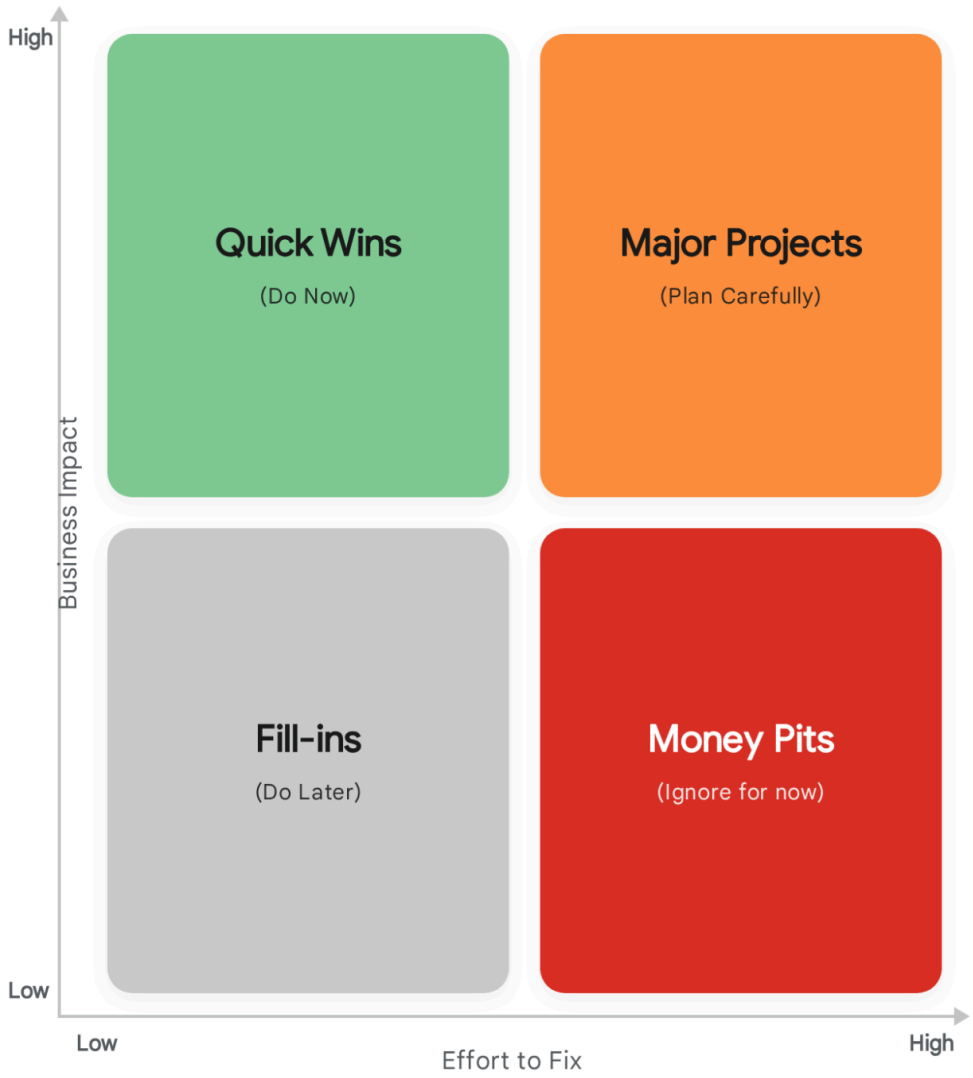
Focus: Pillar 5 (Safe Change & Progressive Delivery). Tackling technical debt sequentially without disrupting value delivery.

Lesson Outline & Slide-Building Material

- **Lesson Title:** The Incremental Scalpel: Orchestrating Tech Debt Reduction at Scale.
- **Short Description:** Utilize Claude CLI subagents and Git worktrees to orchestrate safe, multi-file refactoring without blocking feature development or risking catastrophic regressions.
- **Why this belongs here:** Refactoring large enterprise systems is inherently dangerous. Engineers must learn how to segment debt payoff into small, manageable pieces while utilizing advanced AI CLI capabilities to maintain context.
- **Learning Objectives:**
  1. Differentiate between Technical Debt (code level) and Architectural Debt (system level).[25]
  2. Use Claude Code's analysis tools to identify high-complexity hotspots and prioritize them.
  3. Orchestrate parallel AI tasks using Git worktrees and Claude subagents to safely execute refactoring.[39]
- **Slide-Ready Teaching Points:**
  - **Architecture vs. Code Debt:** Architecture debt (wrong boundaries, siloed data) hides in plain sight; Code debt (messy functions, duplicate logic) is easily visible. AI currently excels at fixing code debt; humans must guide architecture debt resolution.[25]
  - **The Anti-Pattern of Rewrites:** The "Big Bang Rewrite" is almost always a failure. Prefer the Strangler Fig pattern, routing traffic incrementally to newly refactored modules.[26]
  - **Parallel Execution:** Claude Code supports Git worktrees, allowing the AI to spin up an isolated environment, perform a refactor, run the test suite, and create a Pull Request without blocking the developer's main terminal session.[39]
  - **Agent Teams:** Subagents allow role-based AI delegation. You can assign one subagent to refactor the database layer and another to update the UI concurrently,

merging the results.[40]
    - **Quantifiable Debt:** Track technical debt using actionable metrics like the SQALE index, cyclomatic complexity, or defect density rather than raw lines of code.[41]
- **Common Anti-Patterns / Failure Smells:**
    - "Refactoring" without a clear business case or quantifiable metric indicating improvement.
    - Using AI to execute a massive, 50-file refactor in a single PR that no human reviewer can adequately comprehend.[26]
    - Blindly trusting AI-generated code without running static analysis, linters, or security scanning.[42]
- **Discussion Prompt:** "Why do so many 'Version 2.0' complete rewrite projects fail to deliver on time, or get cancelled entirely?"
- **Instructor Demo Idea:** Demonstrate Claude Code subagents and worktrees. Open a terminal and run a command to spawn an agent in a background Git worktree to update a deprecated library, while continuing to explore the codebase in the foreground terminal.
- **AI Leverage Notes:** Use prompts like claude "Analyze code and create a technical debt inventory with severity rankings based on cyclomatic complexity..." to automatically prioritize technical debt.[26]
- **Human Judgment Required:** AI tends to over-engineer abstractions (e.g., adding unnecessary interfaces or generic types) if given open-ended refactoring prompts. Engineers must strictly bound the scope of the AI's task.

Technical Debt Prioritization Matrix



Prioritize technical debt by plotting AI-identified code smells according to their business impact and the effort required to remediate them.

Data sources: Dev.to, MetaMindz

Exercises

- **Exercise Title:** The Surgical Strike.
- **Learning Goal:** Use Claude Code's analysis tools to identify a specific technical debt item, prioritize it, and orchestrate a bounded refactor in an isolated Git worktree.
- **Estimated Duration:** 30 minutes.
- **Setup / Prerequisites:** A sample repository with varied levels of code quality. Claude Code CLI configured.
- **Student Instructions:**
  1. Run the inventory prompt: claude "Analyze the backend codebase. Identify the top 3 functions with the highest cyclomatic complexity. Present them in a markdown table with proposed refactoring strategies."
  2. Review the table and select the top offender. Instruct Claude to spawn a worktree to fix it safely: claude "Create a git worktree named 'refactor-hotspot'. In that worktree, refactor [Function Name] to reduce complexity by extracting helper methods. Ensure all tests pass. Create a git commit."
  3. Navigate to the generated worktree, run git diff, and analyze if the AI successfully reduced the complexity without altering the public method signature or failing the test suite.
- **Expected Output:** A separate git branch containing a small, focused commit that improves code structure, fully verified by human review and passing tests.
- **Instructor Notes / Debrief Points:** Highlight how the worktree feature prevents the developer's main working branch from becoming unstable while the AI experiments with refactoring. Emphasize the rule: "Keep changes small."

Day 5: Maturing the AI-SDLC Practice

Focus: Institutionalizing excellence. Scaling from individual AI use to team-wide governance and measurable outcomes.

Lesson Outline & Slide-Building Material

- **Lesson Title:** Scaling Excellence: Governance, DORA, and Agentic Workflows.
- **Short Description:** Move beyond individual productivity to establish team-wide guardrails using Claude Hooks, Custom Skills, and DORA-aligned AI capability models.
- **Why this belongs here:** This is the culmination of the course. Students must leave knowing how to integrate these practices into their organization's permanent workflow and measure the systemic impact of AI adoption.
- **Learning Objectives:**
  1. Apply the DORA 2025 AI Capabilities Model to organizational processes.[44]
  2. Create Claude Custom Skills (.claude/skills) to standardize team workflows and documentation.
  3. Implement Claude Hooks to enforce security and linting prior to AI code commits.[45]
- **Slide-Ready Teaching Points:**
  - **The 7 Capabilities:** DORA 2025 identifies seven AI capabilities for success,

emphasizing "Healthy data ecosystems," "Strong version control," and "Working in small batches" over mere tool adoption.[44]
    - **Automated Governance:** AI tools must be governed. Claude Hooks (e.g., Pre-commit, FileChanged) act as automated supervisors, running linters or security scans automatically before AI edits are finalized in the workspace.[45]
    - **Democratizing Expertise:** Claude Custom Skills (.claude/skills/SKILL.md) allow teams to codify their specific standards (e.g., /generate-adr, /review-security) and share them across the enterprise via Git, ensuring consistent AI behavior.[46]
    - **Measuring Impact:** Productivity metrics must shift from simple outputs (lines-of-code) to systemic DORA metrics (Deployment Frequency, Lead Time, Change Failure Rate, MTTR).[28]
    - **The Debt Warning:** AI Technical Debt is a critical risk: "The code AI writes today becomes the legacy you maintain tomorrow".[48]
- **Common Anti-Patterns / Failure Smells:**
    - Prompting the AI from scratch every single time instead of creating reusable Custom Skills for repetitive tasks.
    - Treating AI output as inherently safe, skipping static application security testing (SAST) and dependency vulnerability scans.[49]
    - Measuring AI success purely by "developer time saved" rather than "business value delivered safely."
- **Discussion Prompt:** "If AI coding assistants write 80% of our enterprise code by 2028, how will we train our junior developers to understand system architecture and debug complex production failures?"
- **Instructor Demo Idea:** Create a simple Claude Skill (.claude/skills/code-review/SKILL.md) that instructs Claude to review code specifically for OWASP Top 10 vulnerabilities. Run the /code-review command on a flawed file to demonstrate standardized AI output.
- **AI Leverage Notes:** AI can generate its own governance scripts. Ask Claude: "Write a pre-commit hook that prevents hardcoded secrets or API keys from being committed, and configure it in .claude/settings.json."
- **Human Judgment Required:** Governance is fundamentally a human responsibility. AI can enforce rules, but senior engineers must define the risk tolerance and architectural principles that the AI is tasked to enforce.

Measuring Operational Confidence (DORA Metrics)

VELOCITY METRIC

Deployment Frequency

Shows how frequently you deploy code to production.

VELOCITY METRIC

Lead Time for Changes

Measures how quickly a code change goes from commit to production.

STABILITY METRIC

Mean Time to Restore

Measures how quickly your team can restore service after a failure.

STABILITY METRIC

Change Failure Rate

Provides a data-driven approach to analyzing and improving performance based on real research.

True software delivery excellence is tracked not by lines of code generated, but by system velocity and stability measured through industry-standard DORA metrics.

Data sources: Stack Overflow Blog, Axify.io

Exercises

- **Exercise Title:** Capstone: Automating the Standard.
- **Learning Goal:** Synthesize the week's learnings by creating a persistent, shareable Claude Skill that enforces a specific "What Good Looks Like" pillar.
- **Estimated Duration:** 40 minutes.
- **Setup / Prerequisites:** Claude Code CLI installed.
- **Student Instructions:**
  1. Identify one technical debt pain point currently affecting your actual enterprise team (e.g., poor PR descriptions, missing API documentation, lack of security reviews, inconsistent logging).
  2. Create a .claude/skills directory in your project root.
  3. Author a new SKILL.md file (e.g., pr-reviewer.md or logger-check.md). Define the

prompt instructions: What context should Claude read? What specific anti-patterns should it look for? What format should the output be in?
4. Test the skill on your local repository by invoking the custom command /[skill-name].
5. Refine the skill based on the AI's output to ensure it is actionable and not overly verbose.

- **Expected Output:** A functional, reusable Claude Skill committed to version control that automates a code quality or documentation standard for the entire team.
- **Instructor Notes / Debrief Points:** Encourage students to share their skills with the class. Discuss how checking these skills into the team's central Git repository democratizes architectural knowledge and standardizes the AI's behavior across all team members, bridging the gap between junior and senior developers.[46]

Reusable Instructor Assets

A. “What Good Looks Like” Scorecard

This scorecard is designed to be executed against enterprise repositories. Teams score themselves from **0 (Poor)** to **3 (Excellent)** across 15 dimensions.

Dimension	What Good Looks Like (Score 3)	What Poor Looks Like (Score 0)	Why It Matters / Debt Revealed
1. Architecture Clarity	ADRs exist; C4 diagrams reflect code; clear service boundaries.	No high-level docs; circular dependencies; "God" classes.	Reveals architectural debt. Makes AI refactoring highly risky due to hidden coupling.
2. Dev Environment	1-click reproducible setup (e.g., DevContainers, Azure azd).	Multi-day manual setup; OS/framework inconsistencies.	Operational friction. Blocks AI from verifying its own code changes.
3. Build Reliability	CI pipeline runs on every PR; deterministically	Manual builds executed from a single developer's	Build debt. Guarantees "works on my machine" syndromes in

	builds artifacts.	laptop.	production.
4. Test Coverage	Characterization tests cover critical business workflows.	Zero tests, or tests that mock internal logic instead of behavior.	Test debt. Prevents safe legacy modernization; AI will introduce silent regressions.
5. Test Data Management	Ephemeral, versioned databases spun up per test run.	Tests run against a shared, mutable staging database.	Data debt. Results in flaky tests, destroying developer trust in the CI pipeline.
6. Secrets Management	100% integration with Azure Key Vault; Managed Identities used.	Passwords/API keys hardcoded directly in repositories.	Security debt. Exposes the enterprise to catastrophic breach.
7. Observability	OpenTelemetry integrated. Distributed tracing across services.	Unstructured text logs on remote VMs.	Observability debt. Makes AI root-cause analysis of production incidents impossible.
8. Dependency Hygiene	Automated SBOM generation; automated library upgrades.	Libraries are 3+ major versions behind; known CVEs ignored.	Supply chain debt. Drastically increases effort for future upgrades and security patches.
9. Code Complexity	Low cyclomatic complexity enforced via static analysis	Massive functions spanning hundreds of lines of code.	Code debt. Directly correlates with increased defect

	gates.		density and hallucinated AI fixes.
10. Release Safety	Zero-downtime deployments via feature flags.	Manual, weekend "big bang" cutovers.	Release debt. Discourages frequent deployments, slowing value delivery.
11. Environment Parity	Local, Dev, Test, and Prod use identical configurations.	Prod uses Azure App Service, Local uses bare metal IIS.	Configuration debt. Bugs appear in production that cannot be replicated locally.
12. Documentation	CLAUDE.md and READMEs are updated synchronously with code.	Documentation is isolated in Confluence and perpetually stale.	Knowledge debt. AI agents lack the necessary constraints to operate autonomously.
13. Modularity	Code is decoupled; interfaces define clear data contracts.	Monolithic tightly coupled data access and presentation layers.	Design debt. Prevents the safe extraction of microservices.
14. AI Governance	AI usage is governed by pre-commit hooks and custom skills.	Developers paste sensitive code into public, unmanaged LLMs.	Governance debt. Leads to unpredictable code quality and intellectual property leakage.
15. Deployment Freq.	Code is deployed to production multiple	Deployments happen quarterly or	Agility debt. The organization cannot

	times per week.	semi-annually.	respond to market changes or security threats.

B. Technical Debt Prioritization Matrix

When analyzing AI-generated debt inventories (e.g., via /refactor-suggest), use this rubric to prioritize action.[26] Score each factor 1-5, apply the weight, and sort.

1. **Business Impact (Weight: 35%):** Does this debt cause production incidents, block critical feature development, or pose a severe compliance/security risk? (Higher score = Greater impact).
2. **Effort to Fix (Weight: 25%):** Can this be fixed in a single sprint by one developer, or does it require a multi-month architectural shift across teams? (Higher score = Lower effort/faster fix).
3. **Reliability & Blast Radius (Weight: 20%):** If the refactor fails, does it take down a tertiary internal tool or the core payment processing gateway? (Higher score = Lower blast radius).
4. **AI-Friendliness (Weight: 20%):** Is this isolated code debt (messy logic, easily fixed by AI in a worktree) or architectural debt (flawed data models, requires deep human domain knowledge and coordination)? (Higher score = Highly suitable for AI automation).
    - *Decision:* Prioritize **High Impact, Low Effort, High AI-Friendliness** items as immediate "Quick Wins".[51]

C. Capstone Exercise: The AI-Assisted Turnaround

- **Scenario:** Students are handed a legacy Python Django or.NET MVC repository simulating a real enterprise brownfield application. It has no automated tests, a 500-line monolithic controller/view function, and hardcoded database connection strings.
- **Task:**
    1. Assess the repository using the "What Good Looks Like" Scorecard.
    2. Use the Claude Code CLI to generate Characterization Tests for the monolithic controller, achieving 80%+ branch coverage.
    3. Use Claude Code subagents to refactor the controller into smaller, modular services without breaking the test suite.
    4. Implement OpenTelemetry logging for the new services.
    5. Propose a final Pull Request detailing the changes.
- **Deliverable:** A successfully executing CI pipeline showing passing tests, reduced cyclomatic complexity, OpenTelemetry integration, and no hardcoded secrets.

D. Glossary

- **Agentic Workflow:** An AI paradigm where the model autonomously plans, executes tools, and validates its own work recursively, rather than just generating static text.[52]
- **Characterization Test:** An automated test designed to document the actual, current behavior of a legacy system (including bugs), providing a necessary safety net before refactoring.[19]
- **CLAUDE.md:** A markdown file placed in the repository root that serves as persistent contextual memory and instructional constraints for the Claude Code CLI.[31]
- **DORA Metrics:** Industry-standard metrics for measuring software delivery excellence: Deployment Frequency, Lead Time for Changes, Mean Time to Restore (MTTR), and Change Failure Rate.[28]
- **Git Worktree:** A Git feature that allows multiple working trees to be attached to the same repository, enabling Claude Code to execute parallel refactoring tasks without disrupting the main branch.[39]
- **OpenTelemetry (OTel):** An open-source, vendor-neutral framework for instrumenting, generating, collecting, and exporting telemetry data (metrics, logs, and traces) to backends like Azure Application Insights.[53]
- **Strangler Fig Pattern:** A software migration strategy where a legacy system is incrementally replaced by wrapping it with new services and gradually shifting traffic.[26]

E. Top 10 Non-Negotiables

Regardless of the tech stack (.NET, Python, etc.), these practices must survive organizational debates:

1. Code must build locally in under 15 minutes with no manual dependency installation.
2. Untested legacy code must never be refactored without a characterization test baseline.
3. No secrets, credentials, or API keys may be committed to source control.
4. All telemetry must utilize correlation IDs to trace requests across network boundaries.
5. Database schemas must be version-controlled and applied via automated migrations.
6. AI agents must be bounded by CLAUDE.md context and explicit architectural instructions.
7. Changes must be deployed in small batches (enforcing the rule: one PR = one intent).
8. AI-generated code must be subjected to the same strict static analysis and security gates as human-authored code.
9. Human experts must review and approve AI architectural proposals (utilizing Plan Mode).
10. Rollback procedures must be tested, documented, and automated.

Suggested GitHub Repositories and Code Examples

To anchor the theoretical concepts, instructors should reference these tangible examples during

the course to demonstrate enterprise patterns:

1. **Architecture & DevOps Baseline (.NET):**
   - *Repo:* dotnet-architecture/eShopOnWeb [54]
   - *Demonstrates:* Clean architecture principles in.NET, Docker containerization, and GitHub Actions CI/CD workflows. Ideal for evaluating "What Good Looks Like."
2. **Observability (OpenTelemetry in.NET):**
   - *Repo:* nazarii-piontko/sample-dotnet-otel [55]
   - *Demonstrates:* A.NET 8 project with cleanly integrated OpenTelemetry. Students should inspect how telemetry is initialized and routed to external monitors.
3. **Modern Python Architecture (FastAPI):**
   - *Repo:* fastapi-practices/fastapi-best-architecture [56]
   - *Demonstrates:* Multi-tier architecture, dependency injection, and proper project structuring in Python, avoiding the "spaghetti" patterns common in legacy scripts.
4. **Claude Code Extensibility (Skills & Hooks):**
   - *Repo:* DavidROliverBA/Daves-Claude-Code-Skills [46]
   - *Demonstrates:* Real-world implementations of .claude/skills and hooks. Instructors should highlight the /c4-diagram and /adr generation skills to show how AI can automate documentation seamlessly.[10]
5. **Reproducible Cloud Environments:**
   - *Repo:* Azure/azure-dev (Awesome AZD Templates) [57]
   - *Demonstrates:* Using Azure Developer CLI (azd) to orchestrate Infrastructure as Code (Bicep/Terraform) alongside application code, achieving Pillar 2 (Reproducibility) perfectly.
6. **Refactoring Kata (Testing Baseline):**
   - *Repo:* emilybache/GildedRose-Refactoring-Kata [38]
   - *Demonstrates:* The ultimate legacy code playground. Instructors should use this repo for Day 2 to teach characterization testing before allowing AI to refactor the notoriously complex nested logic.

Suggested Readings and Online Resources

Provide these resources to students as pre-reading or post-class deep dives:

1. **State of AI-assisted Software Development (DORA 2025 Report)**
   - *Source:* Google Cloud / DORA [2]
   - *Focus:* Intermediate/Advanced. Teaches the 7 core capabilities required for AI success (e.g., healthy data ecosystems, small batches). Validates the course thesis that AI is a systemic amplifier.

2. **Characterization Testing: Adding Tests to Legacy Code**
   - *Source:* Mario Cervera Blog (referencing Michael Feathers) [19]
   - *Focus:* Beginner/Intermediate. Deep dive into the algorithm for writing tests that capture actual behavior rather than expected behavior. Crucial reading for Day 2.
3. **Claude Code CLI Official Documentation**
   - *Source:* Anthropic [30]
   - *Focus:* Beginner. The primary reference for CLI commands, permission models, agent teams, and setting up CLAUDE.md.
4. **Observability Agent Best Practices for Azure**
   - *Source:* Microsoft Learn [6]
   - *Focus:* Intermediate/Advanced. Details how to properly instrument applications so that AI-driven investigation tools (like Azure Copilot) can successfully analyze logs and traces.
5. **Technical Debt in the AI Era: When Your Assistant Becomes Your Liability**
   - *Source:* Dev.to / Rakbro [48]
   - *Focus:* Advanced. Explores how AI generates plausible but unreliable code, resulting in new forms of "Model Obsolescence Debt" and the psychological traps of automation bias.
6. **Eliminating Technical Debt with Claude Code: A Practical Guide**
   - *Source:* Dev.to / myougatheaxo [26]
   - *Focus:* Intermediate. A highly practical guide detailing the exact workflow for using Claude to inventory debt via cyclomatic complexity analysis and applying the Strangler Fig pattern safely.

Open Questions / Debates / Tradeoffs

The course must acknowledge that AI-assisted SDLC is a nascent and rapidly evolving field. Instructors should openly discuss these debated areas to foster critical thinking:

- **Trusting AI-Generated Tests:** A significant debate exists around AI test generation. If an AI writes both the feature code and the test code, the tests may simply codify the AI's hallucinations or poor logic.[42] *Recommendation:* Enforce strict characterization testing of *existing* code to lock behavior, and mandate human review of all AI-generated assertions. Adopt a "vibe, then verify" culture backed by static analysis.[42]
- **Security vs. Autonomy:** Giving an AI CLI tool full filesystem and execution permissions (e.g., bypassing permission prompts or using Auto-mode) allows for massive productivity but introduces profound security risks, such as executing malicious packages, destabilizing systems via root access, or exposing secrets.[61] *Recommendation:* Utilize strict Claude Hooks to intercept and validate actions, and execute AI agents within sandboxed Git worktrees.[45]

- **Monoliths vs. Microservices for AI Context:** Highly decoupled microservices have historically been viewed as superior enterprise architecture. However, AI agents perform better when they can ingest the entire systemic context at once. Overly fragmented repositories can hinder an AI's ability to reason across boundaries.[62] *Recommendation:* Utilize multi-repository workspaces carefully, ensuring CLAUDE.md files contain explicit pointers to downstream data contracts and APIs.
- **The "Toil Shift":** Does AI actually save time? The DORA report suggests that while raw coding time decreases, the time required to review, audit, and securely integrate AI-generated code increases—shifting the bottleneck from creation to validation.[1] *Recommendation:* Invest heavily in automated CI pipelines (Pillars 2 and 5) to handle the validation burden mechanically, freeing developers to focus on architectural review.

Final Recommendations

To successfully deliver this 5-day course and drive meaningful change within an enterprise software organization, the instructor should adhere to the following directives:

1. **Anchor to Reality, Not Hype:** Continually remind the audience that AI is an execution engine, not a substitute for architectural reasoning. Emphasize that the most valuable AI use case in a brownfield environment is automated understanding (diagramming, mapping, characterizing) rather than automated creation.
2. **Enforce the "Test First" Rule:** Make the Characterization Testing exercise (Day 2) the hardest and most scrutinized gate in the course. Instill the absolute rule that an AI agent is never permitted to alter legacy business logic unless a human-verified test harness surrounds it.
3. **Leverage Azure Native Tooling:** Given the audience's enterprise context, tightly couple the DevOps principles to concrete Azure implementations. Specifically, highlight the synergy between OpenTelemetry, Azure Application Insights, and Azure Developer CLI (azd) to demonstrate a complete, modern enterprise workflow.
4. **Standardize via Hooks and Skills:** Do not let students leave the course relying purely on ad-hoc chat-based interactions. The true enterprise value of the Claude CLI lies in its extensibility. Ensure every student understands how to commit .claude/skills and settings.json (hooks) to the team repository, transforming personal AI habits into enforced organizational standards.
5. **Measure What Matters:** Conclude the course by cementing the DORA metrics. Teach engineering leaders that the success of this AI-SDLC implementation will be judged by reductions in Lead Time and Mean Time to Recovery, not by the sheer volume of code the AI generates.

Works cited

1. Balancing AI tensions: Moving from AI adoption to effective SDLC use - Dora.dev, accessed April 20, 2026, https://dora.dev/insights/balancing-ai-tensions/
2. How are developers using AI? Inside our 2025 DORA report - Google Blog, accessed April 20, 2026, https://blog.google/innovation-and-ai/technology/developers-tools/dora-report-2025/
3. AI-Powered Development with Claude Code: Workflow Guide, accessed April 20, 2026, https://petronellatech.com/blog/claude-code-cli-guide-ai-powered-development/
4. AI-Powered Legacy Code Refactoring: Implementation Guide, accessed April 20, 2026, https://www.augmentcode.com/learn/ai-powered-legacy-code-refactoring
5. Python Observability Part 1: OpenTelemetry Integration with Azure Monitor - Medium, accessed April 20, 2026, https://devputers.medium.com/python-observability-with-azure-monitor-using-opentelemetry-f15959c71ad2
6. Best practices - Azure Copilot observability agent (preview) - Azure Monitor - Microsoft Learn, accessed April 20, 2026, https://learn.microsoft.com/en-us/azure/azure-monitor/aiops/observability-agent-best-practices
7. Building Trustworthy Software with Stochastic Teammates at Unprecedented Scale © 2026 Ahmed E. Hassan, accessed April 20, 2026, https://agenticse-book.github.io/pdf/AgenticSE_Book.pdf
8. 15 Technical Debt Examples for Enterprise Teams | Augment Code, accessed April 20, 2026, https://www.augmentcode.com/guides/15-technical-debt-examples-for-enterprise-teams
9. C4 Generator: Architecture Diagram Claude Code Skill - MCP Market, accessed April 20, 2026, https://mcpmarket.com/tools/skills/c4-architecture-model-generator
10. DavidROliverBA/Daves-Claude-Code-Skills: Reusable ... - GitHub, accessed April 20, 2026, https://github.com/DavidROliverBA/Daves-Claude-Code-Skills
11. Configure your local environment for deploying Python web apps on Azure - Microsoft Learn, accessed April 20, 2026, https://learn.microsoft.com/en-us/azure/developer/python/configure-python-web-app-local-environment
12. Creating Reproducible Development Environments | by John Rofrano | Nerd For Tech, accessed April 20, 2026, https://medium.com/nerd-for-tech/creating-reproducible-development-environments-fac8d6471f35
13. Python Environment Management Best Practices - Inedo Blog, accessed April 20, 2026, https://blog.inedo.com/python/python-environment-management-best-practices
14. Authenticate Azure-hosted Python apps to Azure resources using a system-assigned managed identity - Microsoft Learn, accessed April 20, 2026, https://learn.microsoft.com/en-us/azure/developer/python/sdk/authentication/system-assigned-managed-identity

15. Introducing AWS Transform custom: Crush tech debt with AI-powered code modernization, accessed April 20, 2026, https://aws.amazon.com/blogs/aws/introducing-aws-transform-custom-crush-tech-debt-with-ai-powered-code-modernization/
16. Getting started with Azure Developer CLI (azd) - Microsoft Community Hub, accessed April 20, 2026, https://techcommunity.microsoft.com/blog/startupsatmicrosoftblog/getting-started-with-azure-developer-cli-azd/4392010
17. Using Claude Code: How AI can transform your approach to technical debt - Ada's AI Agent, accessed April 20, 2026, https://www.ada.cx/blog/using-claude-code-how-ai-can-transform-your-approach-to-technical-debt/
18. The 8 software quality metrics that actually matter - DX, accessed April 20, 2026, https://getdx.com/blog/software-quality-metrics/
19. Characterization testing: adding tests to legacy code - Mario Cervera's Blog, accessed April 20, 2026, https://mariocervera.com/characterization-testing-adding-tests-to-legacy-code
20. Can AI-based test generation help you test Legacy Code?, accessed April 20, 2026, https://understandlegacycode.com/blog/ai-test-generation-for-legacy-code/
21. Observability in Azure: A Practical Guide for Developers and Architects | by SOORAJ. V, accessed April 20, 2026, https://medium.com/@v4sooraj/observability-in-azure-a-practical-guide-for-developers-and-architects-a869ce5cfef4
22. Architecture strategies for designing a monitoring system - Microsoft Azure Well-Architected Framework, accessed April 20, 2026, https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/observability
23. The Ultimate Guide to Claude Code CLI: Revolutionizing AI-Assisted Development, accessed April 20, 2026, https://skywork.ai/skypage/en/claude-code-cli-ai-development/2044624815150268416
24. 10 Key Metrics For Software Engineering Leaders to Track - Jellyfish, accessed April 20, 2026, https://jellyfish.co/library/metrics-in-software-engineering/
25. Architecture Debt vs Technical Debt: Why Companies Confuse Them and What It Costs Business, accessed April 20, 2026, https://www.architectureandgovernance.com/uncategorized/architecture-debt-vs-technical-debt-why-companies-confuse-them-and-what-it-costs-business/
26. Eliminating Technical Debt with Claude Code: A Practical Guide - DEV Community, accessed April 20, 2026, https://dev.to/myougatheaxo/eliminating-technical-debt-with-claude-code-a-practical-guide-4e2c
27. Claude Code: The Complete Guide to CLI, Desktop, Web, and GitHub Integration | Elegant Software Solutions, accessed April 20, 2026, https://www.elegantsoftwaresolutions.com/blog/claude-code-complete-guide-cli-desktop-web

28. The four engineering metrics that will streamline your software delivery - Stack Overflow, accessed April 20, 2026, https://stackoverflow.blog/2021/11/29/the-four-engineering-metrics-that-will-streamline-your-software-delivery/
29. AI Code Review Automation: Complete Guide 2025 - Digital Applied, accessed April 20, 2026, https://www.digitalapplied.com/blog/ai-code-review-automation-guide-2025
30. Best Practices for Claude Code - Claude Code Docs, accessed April 20, 2026, https://code.claude.com/docs/en/best-practices
31. The Complete Claude Code Power User Guide: Slash Commands, Hooks, Skills & More, accessed April 20, 2026, https://medium.com/@neonmaxima/the-complete-claude-code-power-user-guide-slash-commands-hooks-skills-more-6a3de2d841cc
32. Software Maturity Models for Startup Scalability - Medium, accessed April 20, 2026, https://medium.com/@wearecapicua/software-maturity-models-for-startup-scalability-a56289e3ed26
33. Spec-Grounded Modernization: Leveraging AI Specification Kits for Brownfield Software Systems | by Bhagyesh Chokhawala | KAIRI | Medium, accessed April 20, 2026, https://medium.com/kairi-ai/spec-grounded-modernization-leveraging-ai-specification-kits-for-brownfield-software-systems-e69bdaf04e32
34. How Claude Code works - Claude Code Docs, accessed April 20, 2026, https://code.claude.com/docs/en/how-claude-code-works
35. How to refactor 50k lines of legacy code without breaking prod using claude code - Reddit, accessed April 20, 2026, https://www.reddit.com/r/ClaudeCode/comments/1qobg1g/how_to_refactor_50k_lines_of_legacy_code_without/
36. Best Practices for AI Refactoring Legacy Code: 7 Safe Rules - CodeGeeks Solutions, accessed April 20, 2026, https://www.codegeeks.solutions/blog/best-practices-for-ai-refactoring-legacy-code
37. TDD workflows with Claude Code - what's actually working after months of iteration (Staff eng, w/ 14 yrs exp) : r/ClaudeCode - Reddit, accessed April 20, 2026, https://www.reddit.com/r/ClaudeCode/comments/1qd64xx/tdd_workflows_with_claude_code_whats_actually/
38. Starting code for the GildedRose Refactoring Kata in many programming languages. - GitHub, accessed April 20, 2026, https://github.com/emilybache/GildedRose-Refactoring-Kata
39. Introduction to Git Worktrees in Claude Code | Better Stack Community, accessed April 20, 2026, https://betterstack.com/community/guides/ai/git-worktrees-claude/
40. Claude Code's Agent Teams IS INSANE! Deploy A Full AI Engineering Team! Multiple AI Agents Coding!, accessed April 20, 2026, https://www.youtube.com/watch?v=6UKUQNcRk2k

41. 9 Enterprise Strategies to Slash Technical Debt | Augment Code, accessed April 20, 2026, https://www.augmentcode.com/guides/9-enterprise-strategies-to-slash-technical-debt
42. The great toil shift: How AI is redefining technical debt - Sonar, accessed April 20, 2026, https://www.sonarsource.com/blog/how-ai-is-redefining-technical-debt
43. Automated Technical Debt Detection with Claude Code /refactor-suggest - DEV Community, accessed April 20, 2026, https://dev.to/myougatheaxo/automated-technical-debt-detection-with-claude-code-refactor-suggest-9hi
44. State of DevOps 2025: Review of the DORA Report on AI Assisted Software Development, accessed April 20, 2026, https://www.splunk.com/en_us/blog/learn/state-of-devops.html
45. Automate workflows with hooks - Claude Code Docs, accessed April 20, 2026, https://code.claude.com/docs/en/hooks-guide
46. Skills and Hooks Starter Kit for Claude Code, accessed April 20, 2026, https://medium.com/@davidroliver/skills-and-hooks-starter-kit-for-claude-code-c867af2ace32
47. Extend Claude with skills - Claude Code Docs, accessed April 20, 2026, https://code.claude.com/docs/en/skills
48. Technical Debt in the AI Era: When Your Assistant Becomes Your Liability - DEV Community, accessed April 20, 2026, https://dev.to/rakbro/technical-debt-in-the-ai-era-when-your-assistant-becomes-your-liability-3bd2
49. AI Is Creating Technical Debt – How Enterprises Should Handle It - SD Times, accessed April 20, 2026, https://sdtimes.com/ai/ai-is-creating-technical-debt-how-enterprises-should-handle-it/
50. AI-Assisted Technical Debt Register: Prioritize, Score, and Pay Down Faster - Routine, accessed April 20, 2026, https://routine.co/blog/posts/ai-technical-debt-prioritize
51. How to track and prioritize technical debt - TinyMCE, accessed April 20, 2026, https://www.tiny.cloud/blog/technical-debt-tracking/
52. Use These AI Workflows To Reduce Your Technical Debt - The New Stack, accessed April 20, 2026, https://thenewstack.io/use-these-ai-workflows-to-reduce-your-technical-debt/
53. Application Insights OpenTelemetry observability overview - Azure Monitor | Microsoft Learn, accessed April 20, 2026, https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview
54. dotnet-architecture/eShopOnWeb: Sample ASP.NET Core 8.0 reference application, now community supported: https://github.com/NimblePros/eShopOnWeb · GitHub - GitHub, accessed April 20, 2026, https://github.com/dotnet-architecture/eShopOnWeb
55. nazarii-piontko/sample-dotnet-otel: Sample ASP.NET (.NET 8) project with OpenTelemetry integration - GitHub, accessed April 20, 2026,

https://github.com/nazarii-piontko/sample-dotnet-otel
56. fastapi-practices/fastapi-best-architecture: Enterprise-level backend architecture solution with fastapi、sqlalchemy,、celery、pydantic、grafana、docker... · GitHub - GitHub, accessed April 20, 2026, https://github.com/fastapi-practices/fastapi-best-architecture
57. Awesome Azure Developer CLI, accessed April 20, 2026, https://azure.github.io/awesome-azd/
58. Kickstart projects with azd Templates - DEV Community, accessed April 20, 2026, https://dev.to/azure/kickstart-projects-with-azd-templates-3kjn
59. Introducing the DORA AI Capabilities Model: 7 keys to succeeding in AI-assisted software development - Google Research, accessed April 20, 2026, https://research.google/pubs/introducing-the-dora-ai-capabilities-model-7-keys-to-succeeding-in-ai-assisted-software-development/
60. Claude Code overview - Claude Code Docs, accessed April 20, 2026, https://code.claude.com/docs/en/overview
61. Engineering Pitfalls in AI Coding Tools: An Empirical Study of Bugs in Claude Code, Codex, and Gemini CLI - arXiv, accessed April 20, 2026, https://arxiv.org/html/2603.20847
62. Top AI Coding Tools 2025 for Enterprise Developers, accessed April 20, 2026, https://www.augmentcode.com/tools/top-ai-coding-tools-2025-for-enterprise-developers